

# Erciyes Üniversitesi

## Bilgisayar Mühendisliği Bölümü

---

BZ 313 Yazılım Mühendisliği

8. Gereksinimler için  
Kullanılabilir Modeller

# Gereksinimler için Modeller

---

Bakış açısı analizi, senaryolar, use-case'ler vb. aracılığıyla gereksinimleri anlamaya çalışırken veya gereksinimleri analiz etmek ve belirtmek için **modeller** kullanılır.

Modeller, müşterinin anlayışı ile geliştiricilerin anlayışı arasında bir köprü sağlar.

**Gereksinim analizi ve spesifikasyonların belirlenmesinde belirli bir görev için uygun aracın seçilmesi gerekir.**

- Çeşitli araçlar ve teknikler seçilebilir.
- Bunlardan bazıları diğer derslerden aşına olduğunuz tekniklerdir.
- Her duruma uyan doğru bir teknik bulunmaz.

# Modeller: Faydalı Yayınlar

---

Grady Booch, James Rumbaugh, Ivar Jacobson, *The Unified Modeling Language*. Addison-Wesley 1999.

Grady Booch, et al., *Object-Oriented Analysis and Design with Applications*, third edition. Benjamin/Cummings 2007.

Rob Pooley, Perdita Stevens, *Using UML Software Engineering with Objects and Components*, second edition. Addison-Wesley 2005.

# Modeller

---

## Model, gerçekliğin basitleştirilmesidir

- Geliştirmekte olduğumuz sistemi daha iyi anlayabilmemiz için modeller oluşturuyoruz.
- Karmaşık sistem modelleri inşa ederiz çünkü böyle bir sistemi bütünüyle kavrayamayız.

Modeller gayri resmi veya resmi olabilir. Proje ne kadar karmaşıkça, resmi bir model o kadar değerli hale gelir.

# Modelleme İlkeleri

---

- Model seçimi, sorunu çözme yaklaşımımızı ve çözümün şeklini belirler.
- **Tek bir model** yeterli değildir.
  - Karmaşık sistemler, genellikle birden fazla modelin birlikte kullanılmasıyla daha iyi anlaşılır.
- Modeller farklı ayrıntı düzeylerinde ifade edilebilir.
- Ayrıntı düzeyinden bağımsız olarak, iyi modeller gerçekliği yansıtmalıdır.

# Unified Modeling Language

---

**UML, yazılım sistemlerini modellemek için standart bir dildir**

- Gereksinimler ve uygulama arasında bir köprü görevi görür.
- Bir yazılım sisteminin tasarımını belirtmek ve belgelemek için bir araçtır.
- Süreç ve programlama dilinden bağımsız olması amaçlanmıştır, ancak özellikle nesne yönelimli program geliştirme için uygundur.

# Rational Rose

---

Rational Rose, UML modelleri (diyagramlar ve belirtilimler) oluşturmak ve yönetmek için IBM'e ait bir sistemdir.

Bu ders kapsamında açık kaynaklı MODELIO uygulaması kullanılmıştır.

# Modeller: UML'de Diyagramlar ve Spesifikasyonlar

---

UML'de bir model; bir **diyagram** ve bir **spesifikasyondan** oluşur.

- **Diyagram**, genellikle köşelerin (nesnelerin) ve yayların (ilişkilerin) bağlantılı bir grafiği olarak işlenen bir dizi ögenin grafiksel temsilidir.
- Her diyagram, diyagramın temsil ettiği modeli daha ayrıntılı olarak **belirten teknik belgelerle** desteklenir.

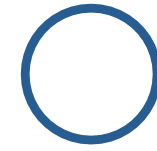
Spesifikasyonu olmayan bir diyagram çok işe yaramayacaktır.

| Kullanici                     |
|-------------------------------|
| + girisYap()                  |
| ad : string<br>email : string |
|                               |

# Veri Akışı Modelleri

---

Bir sistem üzerinden veri akışını göstermek için resmi olmayan bir modelleme tekniği.



Harici varlıklar



İşleme adımları



Veri depoları veya kaynakları

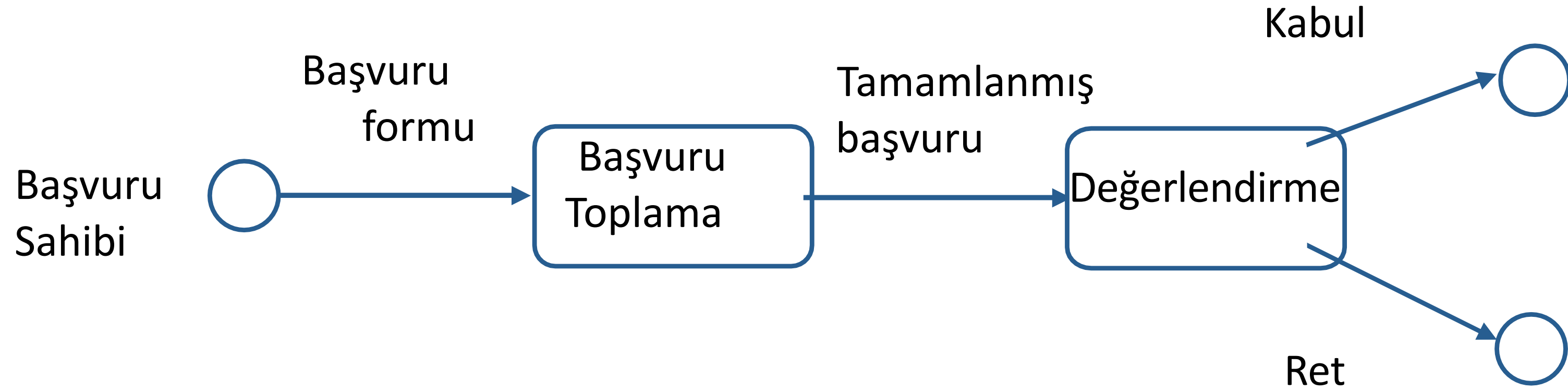


Veri akışları

# Veri Akışı Modeli

## Örnek: Özel Okul Kabulleri (İlk Deneme)

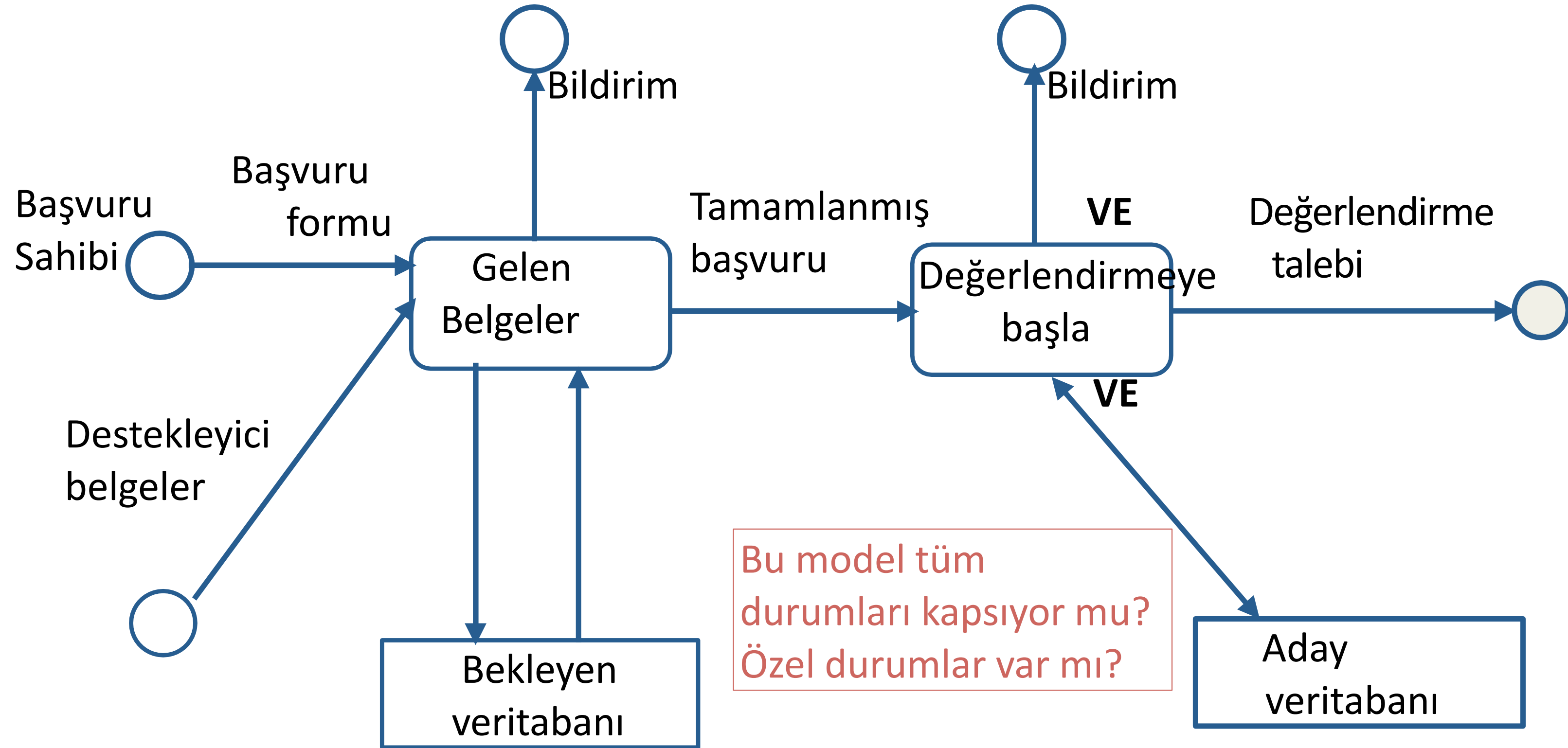
---



Akışı gösterir, ancak veriler nerede depolanır? Destekleyici bilgi var mı?

# Veri Akışı Modeli

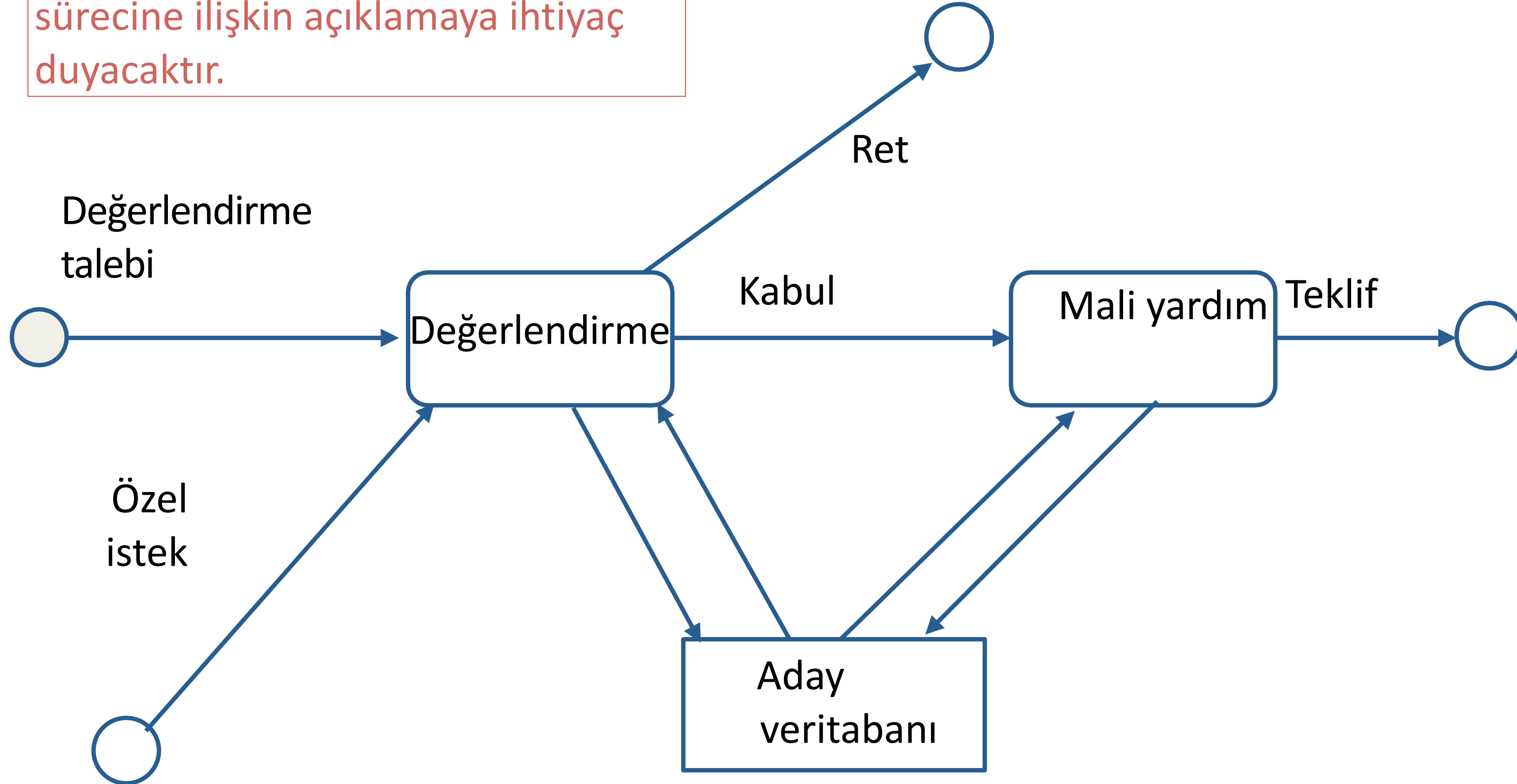
## Örnek: Başvuru Toplama



# Veri Akışı Modeli

## Örnek: Tamamlanan Başvuru

Gereksinimler, karar verme sürecine ilişkin açıklamaya ihtiyaç duyacaktır.



# Karar Tablosu (Decision Table) Modeli

## Üniversiteye Kabul Kararı

|                       |   |   |   |   |   |   |
|-----------------------|---|---|---|---|---|---|
| SAT > S1              | T | F | F | F | F | F |
| GPA > G1              | - | T | F | F | F | F |
| SAT between S1 and S2 | - | - | T | T | F | F |
| GPA between G1 and G2 | - | - | T | F | T | F |
| <i>Accept</i>         | X | X | X |   |   |   |
| <i>Reject</i>         |   |   |   | X | X | X |

Her sütun ayrı bir karar durumudur. Sütunlar soldan sağa doğru işlenir.

Kuralların spesifik ve test edilebilir olduğunu unutmayın.

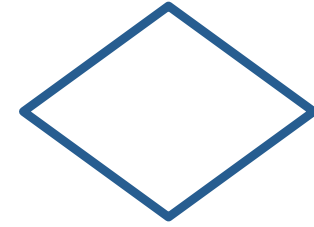
# Akış Şeması Modelleri

---

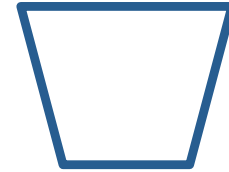
Bir sistemin bir parçasının mantığını ve verilerin bir sistem boyunca izlediği yolları göstermek için kullanılan resmi olmayan bir modelleme tekniği.



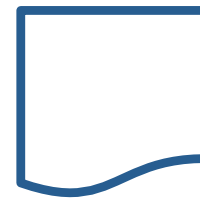
İşlem



Karar



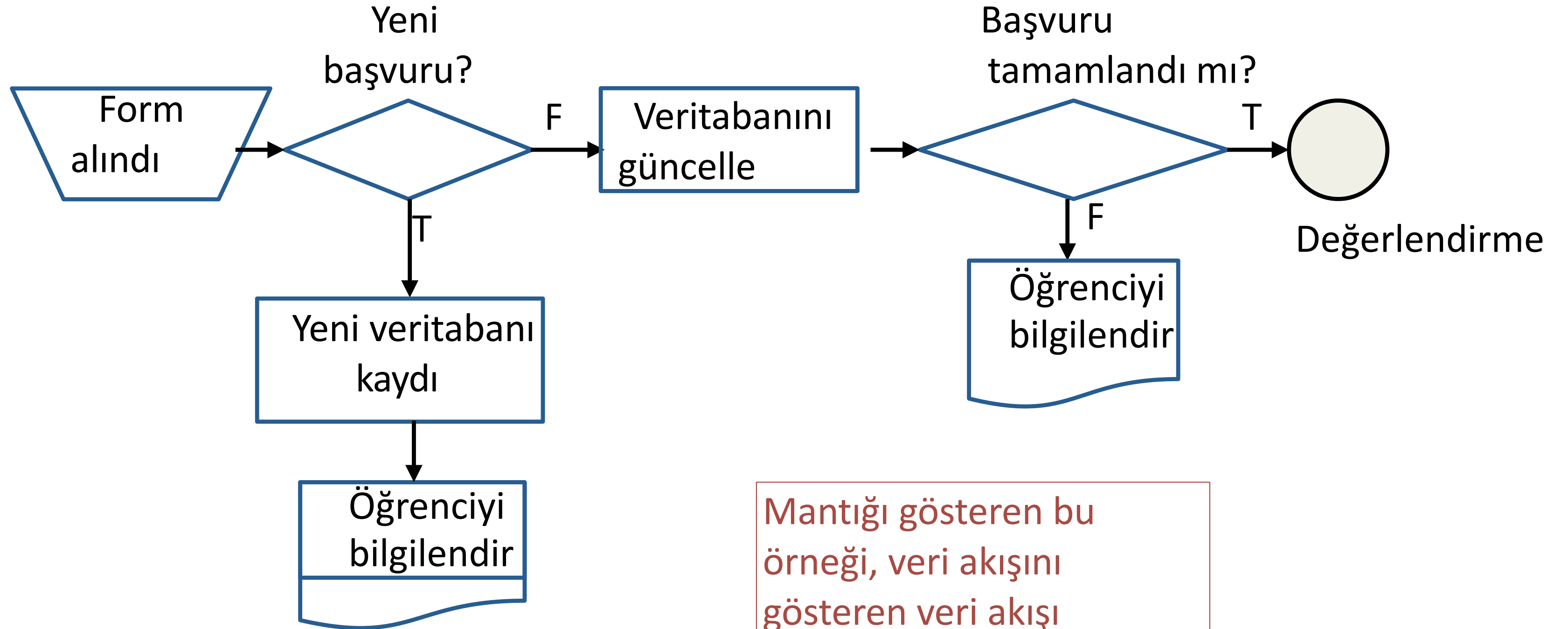
Manuel işlem



Rapor

# Akış Şeması Modeli

## Örnek: Üniversite Kabul Toplantısı



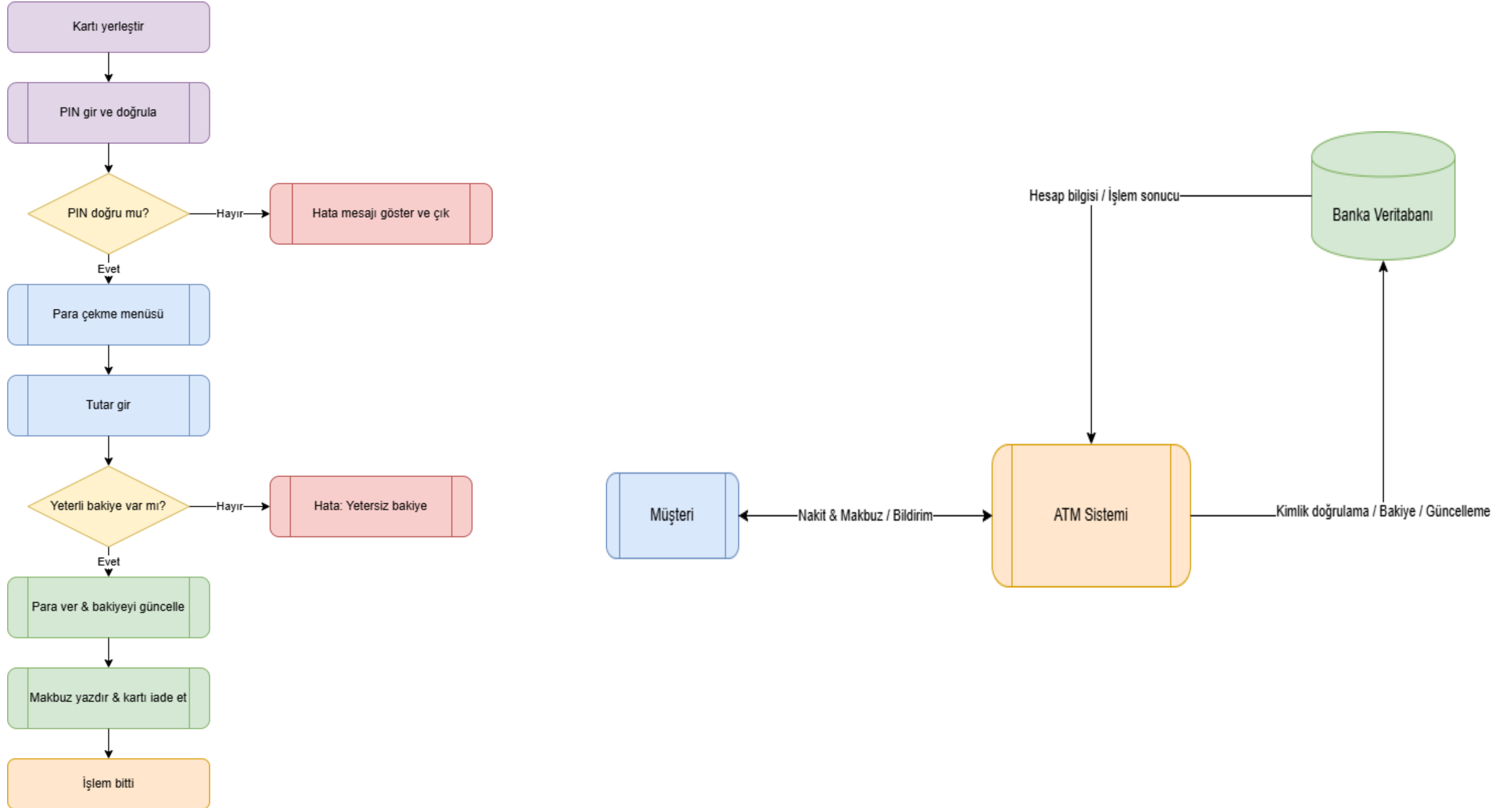
Mantığı gösteren bu örneği, veri akışını gösteren veri akışı modeliyle karşılaştırın.

# ÖRNEK

---

"ATM Para Çekme" için hem **Veri Akış Modeli** hem **Akış Şeması** çizin.

# Akış Şeması / Veri Akış Şeması



# Modelleme Araçları: Sözde kod (Pseudo-code)

---

Bir sistemin bir parçasının arkasındaki mantığı göstermek için gayri resmi bir modelleme tekniğidir.

## Örnek: Üniversiteye Kabul Kararı

**admin\_decision** (application)

```
if application.SAT == null then error (incomplete)
if application.SAT > S1 then accept(application)
else if application.GPA > G1 then accept(application)
else if application.SAT > S2 and application.GPA > G2
    then accept(application)
else reject(application)
```

Sözde kod için kullanılan gösterim gayri resmi veya bir yazılım geliştirme kuruluşu tarafından kullanılan bir standart olabilir veya normal bir programlama diline dayalı olabilir. Önemli olan, yorumunun ilgili herkes tarafından anlaşılmasıdır.

# Sözde Kod Örnekleri

---

## Genel

- Öğrenci sisteme giriş yapar
- Sistem öğrenciyi doğrular
- Başarılı ise ana sayfayı göster

## İdeal

```
admin_decision (application)
    if application.SAT == null then
        return error("incomplete application")
    if application.SAT > S1 then
        return accept(application)
    else if application.GPA > G1 then
        return accept(application)
    else if application.SAT > S2 and application.GPA > G2
then
    return accept(application)
else
    return reject(application)
```

## Detay

```
def admin_decision(application):
    if application['SAT'] is None:
        raise ValueError("Incomplete application: SAT score
missing")
    elif application['SAT'] > 1400:
        application['status'] = 'accepted'
    return {'result': 'accept', 'application': application}
```

## Örnek

---

Bir e-ticaret sitesinde sepetteki ürünlerin toplam tutarını hesaplayan ve eğer tutar 100 TL'den fazlaysa %10 indirim uygulayan algoritma yazınız.

```
function sepetHesapla(urunler)
    toplam  $\leftarrow$  0
    for her urun in urunler
        toplam  $\leftarrow$  toplam + (urun.fiyat * urun.adet)

    if toplam > 100 then
        indirim  $\leftarrow$  toplam * 0.10
        toplam  $\leftarrow$  toplam - indirim

    return toplam
```

# Modelleme Araçları: Geçiş (Transition) Diyagramları

Sistem  $S_i$  **durum** (states) seti olarak tasarlanır.

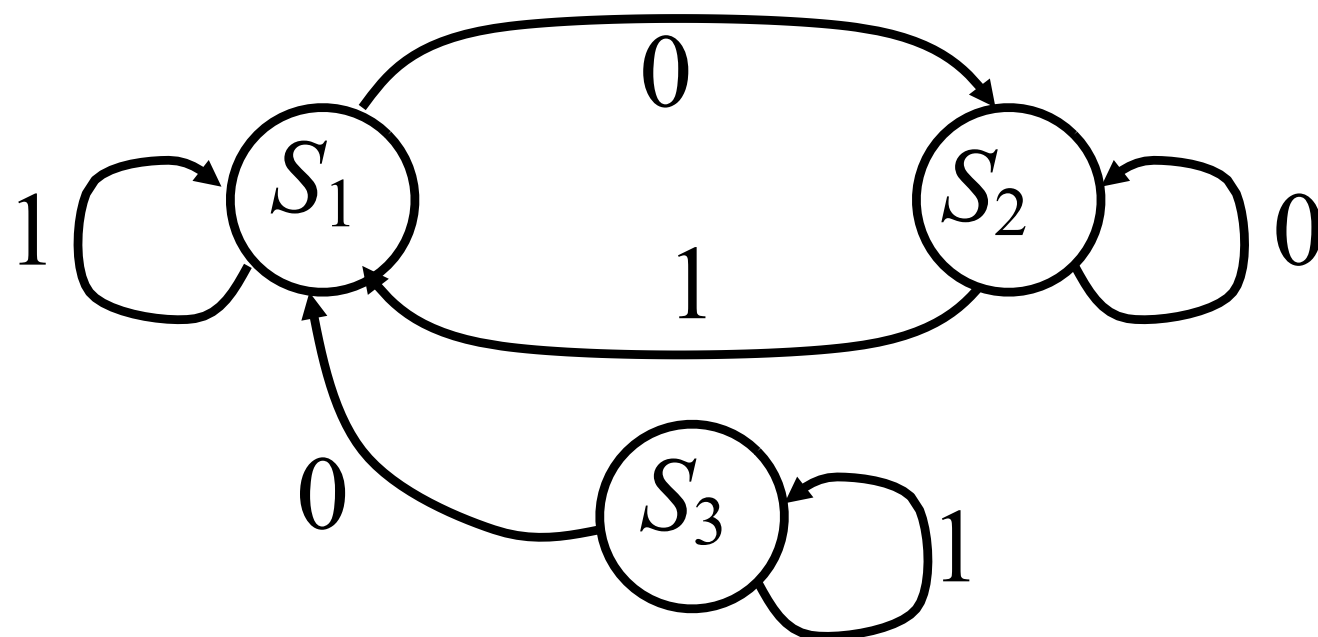
**Geçiş**, bir durumdan diğerine geçiştir.

Bir  $C_i$  **koşulunun** meydana gelmesi ile bir durumdan diğerine geçiş gerçekleşir.

**Geçiş fonksiyonu:**

$$f(S_i, C_j) = S_k$$

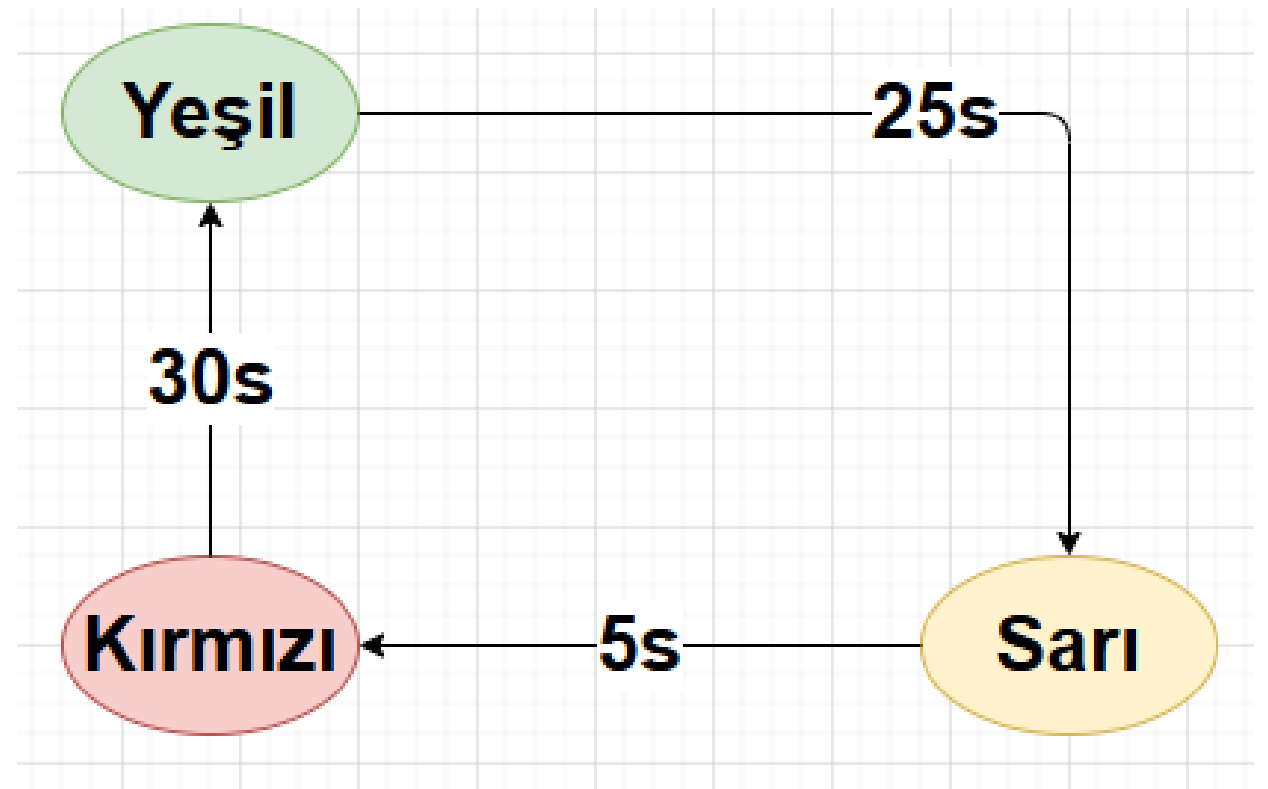
**Örnek**



$f(\text{Kırmızı}, 30s\text{Geçti}) = \text{Yeşil}$

$f(\text{Yeşil}, 25s\text{Geçti}) = \text{Sarı}$

$f(\text{Sarı}, 5s\text{Geçti}) = \text{Kırmızı}$



# Sonlu Durum Makine Modeli

## Terapi Kontrol Konsolu

---

### Örnek: Radyasyon Tedavisi Kontrol Konsolu

Operatörün kontrol konsolu için gereksinimler geliştirdiğinizi varsayalım.

Bir görüşme sırasında müşteri, operatörün makineyi çalıştırırken izlemesi gereken adımları ve prosedürleri açıklamaktadır.

Siz de bu prosedürleri ifade etmek için bir sonlu durum makinesi (finite state machine) modeli kullanmaya karar verirsiniz.

Bu modelin amacı, hem müşteriye gereksinimleri doğru anladığınızı göstermek hem de geliştiriciler için prosedürleri açık bir biçimde tanımlamaktır.

# Sonlu Durum Makine Modeli

## Terapi Kontrol Konsolu: Senaryo

---

### Senaryo

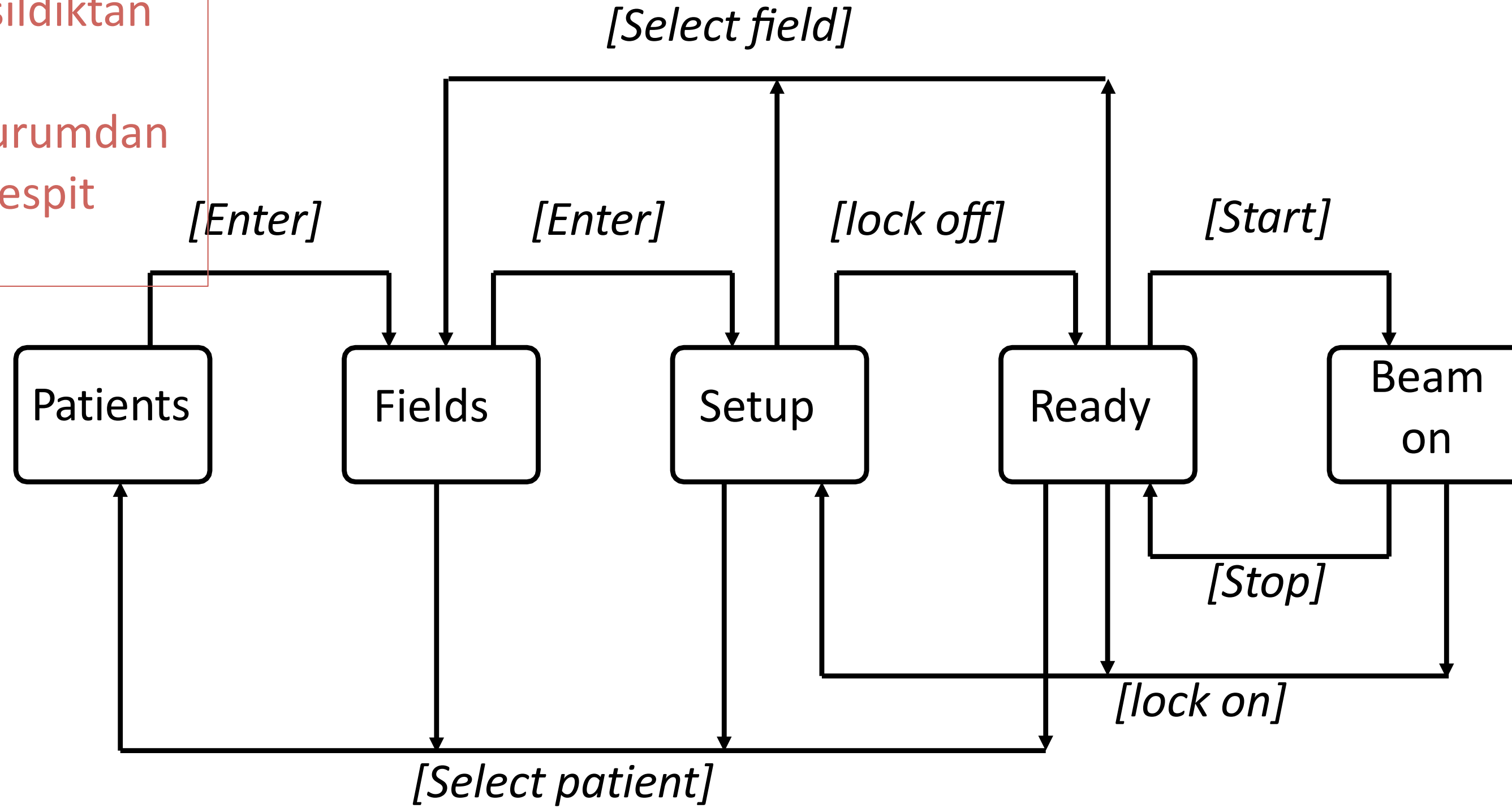
Müşteri aşağıdaki genel senaryoyu paylaşmaktadır:

“Hasta hazır hale getirilmeden önce kurulum yapılır. Operatör, hasta bilgilerini bir veri tabanından seçer. Seçim yapıldığında, ilgili hasta için onaylanmış radyasyon alanlarının bir listesi görüntülenir. Operatör ilk alanı seçer ve böylece kurulum tamamlanır. Hasta artık hazırdır. Kilit makineden çıkarılır ve bu alana uygun dozlar uygulanır. Ardından operatör, alan seçimi ekranına geri dönerek bir sonraki alanı seçer.”

# Sonlu Durum Makine Modeli

## Durum Geçiř Diyagramı

Her durumu ve geçiř müřteriyle tartıřıldıktan sonra operatör konsolunun 5 durumdan birinde olduđu tespit edilir.



# Sonlu Durum Makine Modeli

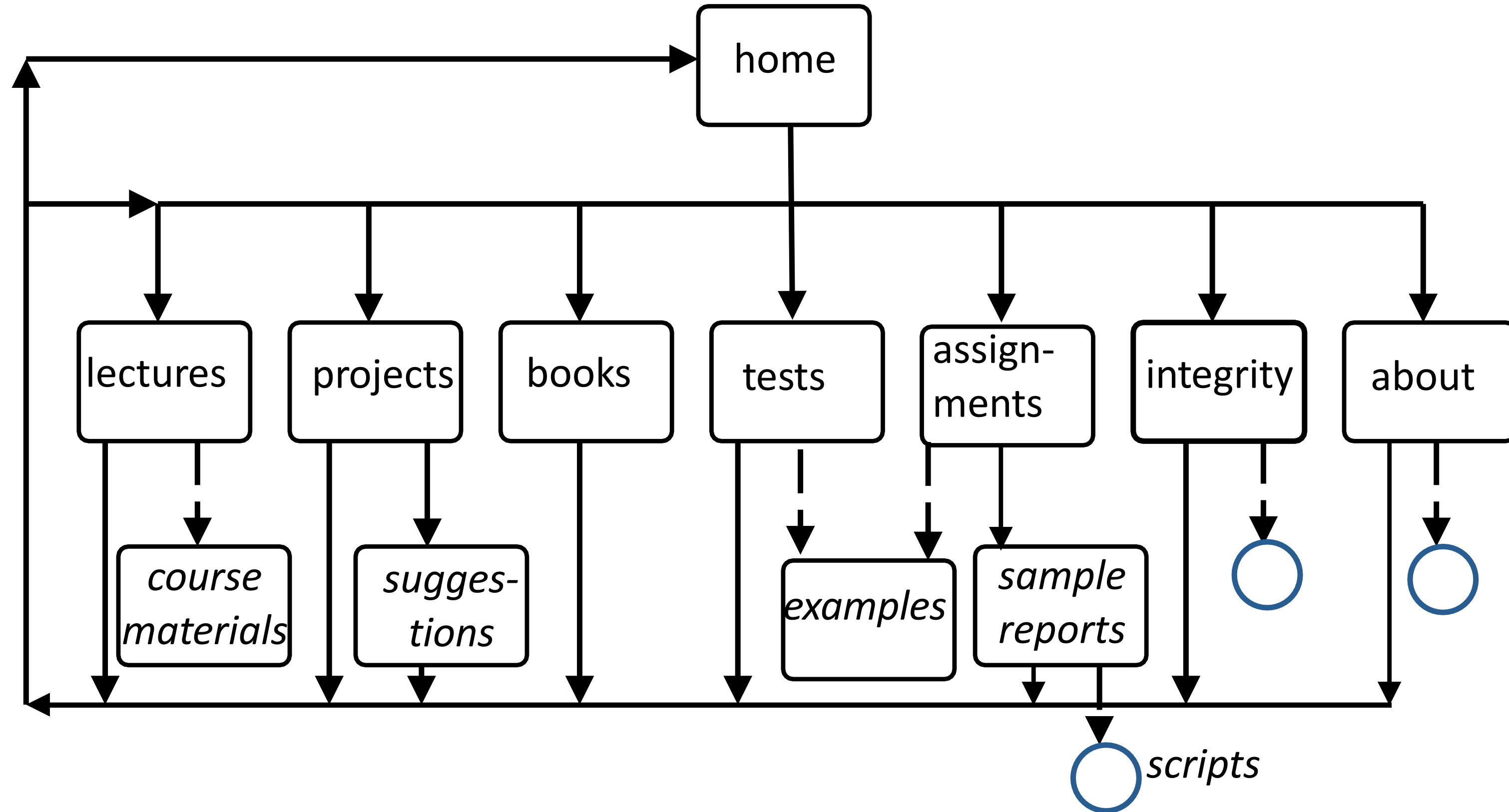
## Durum Geçiş Tablosu

|          | <i>Select Patient</i> | <i>Select Field</i> | <i>Enter</i> | <i>lock off</i> | <i>Start</i> | <i>Stop</i> | <i>lock on</i> |
|----------|-----------------------|---------------------|--------------|-----------------|--------------|-------------|----------------|
| Patients | _____                 | _____               | Fields       | _____           | _____        | _____       | _____          |
| Fields   | Patients              | _____               | Setup        | _____           | _____        | _____       | _____          |
| Setup    | Patients              | Fields              | _____        | Ready           | _____        | _____       | _____          |
| Ready    | Patients              | Fields              | _____        | _____           | Beam on      | _____       | Setup          |
| Beam on  | _____                 | _____               | _____        | _____           | _____        | Ready       | Setup          |

Bu tablo, gereksinimlerin tanımlanması, program tasarımı ve kabul testi için kullanılabilir.

# Kullanıcı Arayüzleri için Geçiş Diyagramı

## Örnek: Bir Web Sayfası



# Varlık-İlişki Modeli

---

**İlişkisel veri tabanlarında gereksinimler ve tasarımlar için standart metodolojidir.**

- Varlıklar ve ilişkilerden oluşan bir veri tabanı
- Varlık-ilişki diyagramlarını görüntülemek ve değiştirmek için araçlar
- Veritabanını işlemek için araçlar (örneğin, veri tabanı tasarımına girdi olarak)

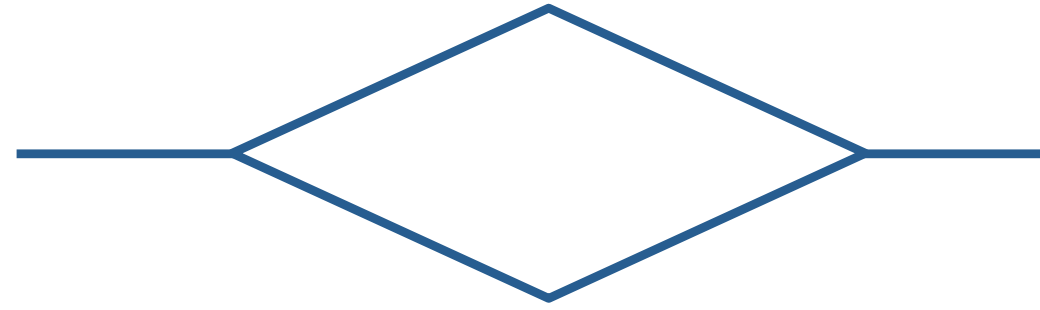
Varlık-ilişki modelleri hem gereksinim belirtimi hem de tasarım belirtimi için kullanılabilir.

# Modelleme Araçları: Varlık-İlişki Diyagramı

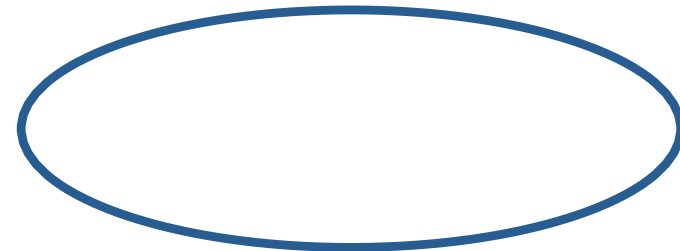
---



Bir varlık (isim)



Varlıklar arasındaki ilişki (fiil)



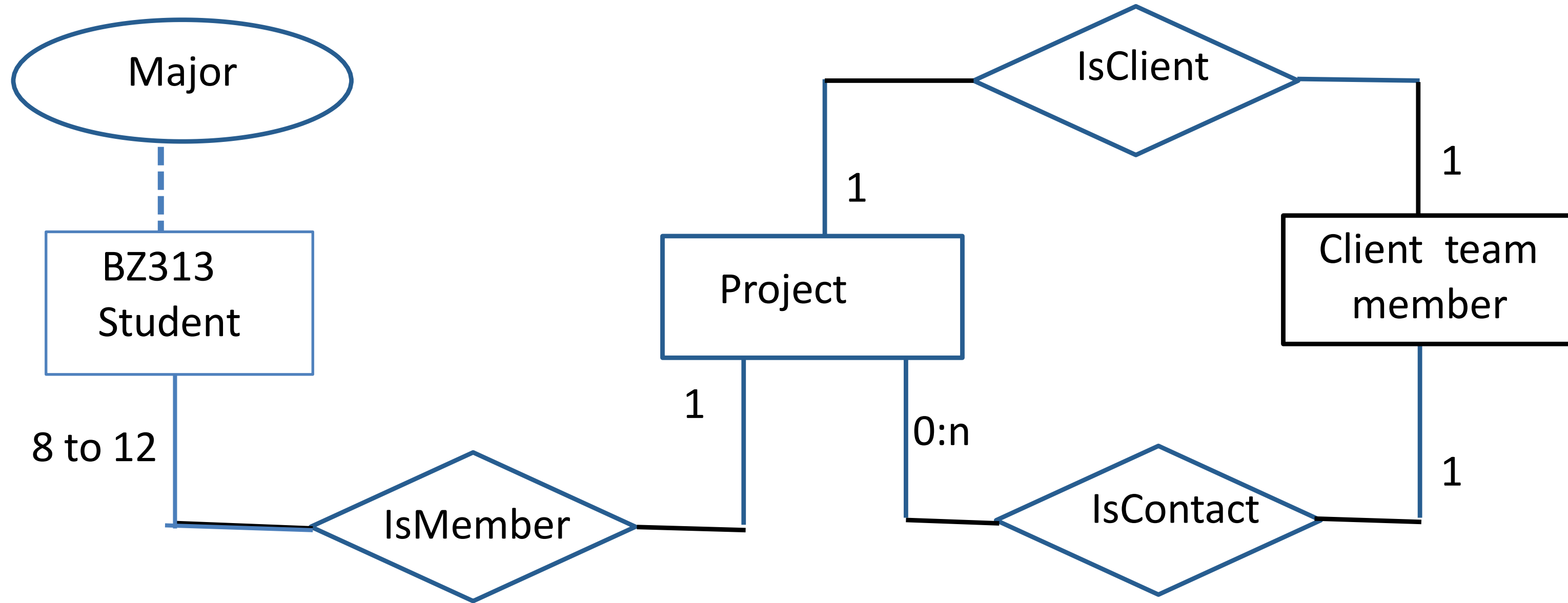
Bir varlık veya ilişki özniteliği

*Not: Varlık-ilişki diyagramları için kullanılan çeşitli gösterimler vardır.  
Bu, Chen (1976) tarafından kullanılan gösterimdir.*

# Modelleme Araçları: Varlık İlişkisi Diyagramı

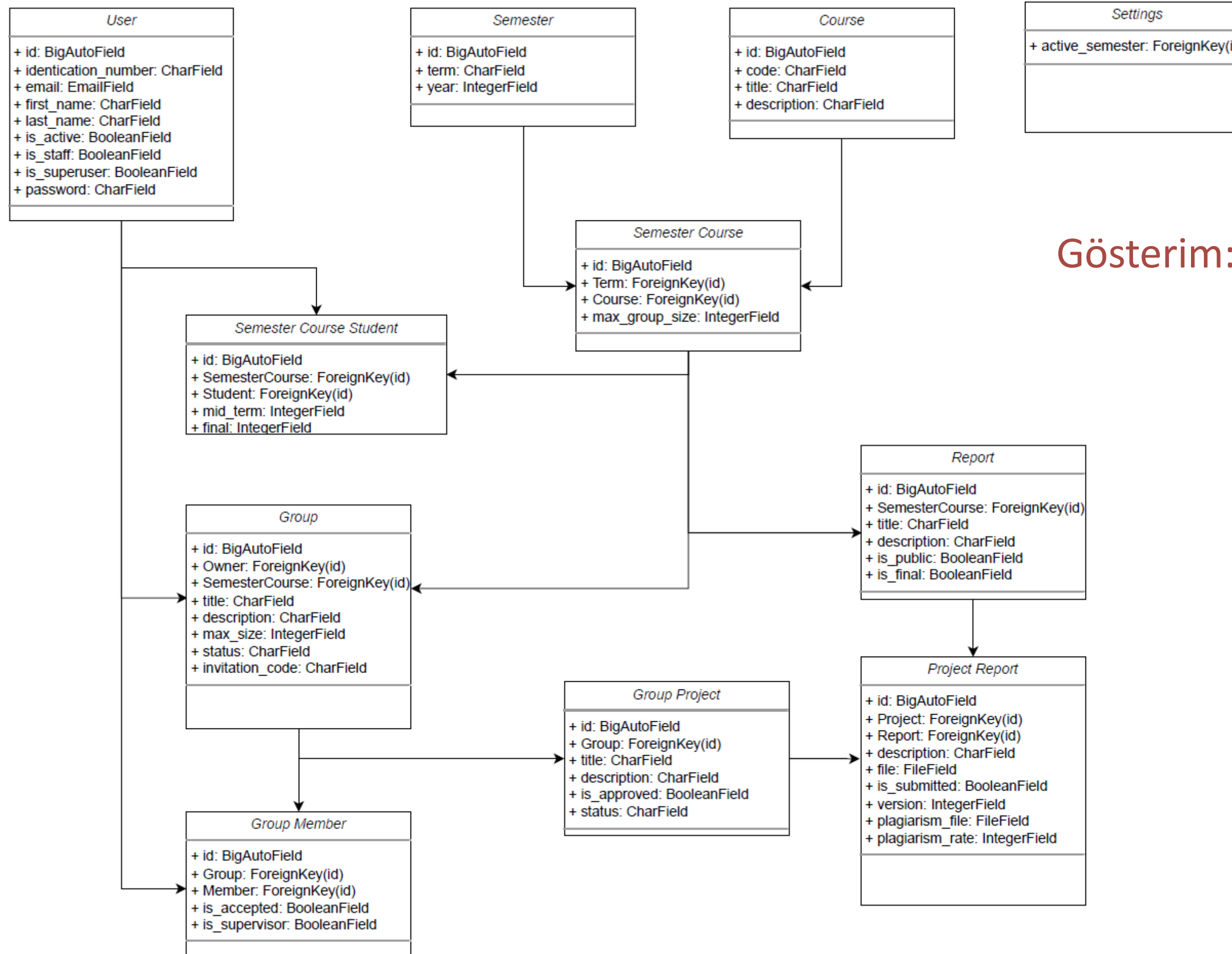
## Örnek:

---



# Bir Tasarım Aracı Olarak Varlık İlişkisi Diyagramı

## Örnek: Web Verileri için Veritabanı Şeması



Gösterim: Her tablo bir varlığı temsil eder  
Her ok bir ilişkiyi temsil eder

# Gereksinimleri Prototipleme

---

**Hızlı prototipleme, tüm modelleme yöntemlerinin en kapsamlısıdır**

Gerekli sistemin temel parçalarının işlevselliğini gösteren bir sistem oluşturarak gereksinimleri belirleme yöntemidir.

Kullanıcı arayüzleri için özellikle değerlidir

# Gereksinim Tanımı: Veri Sözlükleri

---

**Veri sözlüğü, sistem tarafından kullanılan adların bir listesidir**

- Ad (ör., "start\_date")
- Kısa tanım (ör., "date" nedir)
- Nedir? (ör., integer, relation)
- Nerelerde kullanılır (örneğin, kaynak, kullanan, vb.)
- Bir sözlükle birleştirilebilir

Sistem geliştirildikçe gereksinimlerdeki veri sözlüğü, nihai dokümantasyonun bir parçası olabilecek sistem veri sözlüğünün temelini oluşturur.

# Sınıf ve Nesne Modelleri Üzerine Bir Not

---

- **Program tasarımında** oldukça önemli olan **sınıf** ve **nesne** modelleri bazen gereksinim modellemede de önerilmektedir.
- Ancak sistem tasarımını çok fazla kısıtladığı için sınıf ve nesne modellemeyi gereksinim düzeyinde kullanmak zor olabilmektedir.
- **Akış modelleri** ve **sonlu durum makinaları** UML'ler tarafından desteklenmektedir ve **gereksinimleri modellemek** için daha faydalı olabilmektedir.

# Erciyes Üniversitesi

## Bilgisayar Mühendisliği Bölümü

---

BZ 313 Yazılım Mühendisliği

8. Gereksinimler için  
Kullanılabilir Modeller

Ders Sonu